

Feb 11, 2026

CUDA:OpenCL translation - Transcript

00:00:00

Erel Kaplan: Okay. Hi everyone. Uh so uh I'll show my uh my progress in the parcodex path. The main thing for me now to do is to make it app like so everyone can use it uh

Gal Oren: Nice.

Erel Kaplan: easily and I wanted to remove every outside dependencies it had for example like a a JSON file with the code or the API. This is now totally confir configurable by the user. So I'm working on on that and I had some try I I tried some uh some some things to make the token usage of the of the H agent a bit less but I was not very successful. Uh so I'll I'll show you why uh it uses so so many tokens and uh I'll I'll explain why I was not able to make it use less and I'll show you the current situ situation with the app that I'm uh building. Okay. So let me know when you see my screen.

Tomer Bitan: How we

Erel Kaplan: Okay. So this is uh how the how how the the the token

00:01:42

Gal Oren: Now

Erel Kaplan: situation works with the agent. So there's h like the system instructions that uh it comes with the agent itself plus the user instructions and the all prompt and everything. This is basically 20k tokens. And h with with each each turn we see like this little green green part is added but everything is going this is like the total token usage. So here it uses 20k tokens and in the next turn it's already using this 20k plus it moves this entire h starter start starter message from one turn to to to the other. So we we get this plus this. So we have like 40 40 something,000 tokens and each each turn we it goes on and on and on. So the the like per message tokens is not much.

Gal Oren: Nothing.

Erel Kaplan: Um you can see like this is relatively the the the ratios between the the how long is the pair turn uh tokens and the start start message tokens. So you can see that it

is not much but because the entire context is moving from uh one turn to to the other h we get a total of really really high token usage.

00:03:34

Erel Kaplan: Yes. So this is one thing.

Gal Oren: Ariel, do you have um by the way,

Erel Kaplan: Yeah.

Gal Oren: do you have uh even like a one showcase of the difference in the total amount of tokens per uh model usage.

Tomer Bitan: Wait,

Erel Kaplan: Yeah.

Gal Oren: Let's say that you're fixing on the minimum and now you have 5.1,

Erel Kaplan: Yeah. Yeah. Yeah.

Gal Oren: 5.2 and 5.3.

Erel Kaplan: Yeah. Yeah. So what what I was comparing here is the I I have here in the next slides I have this is Yeah.

Gal Oren: The next

Tomer Bitan: can I can I ask a question about that or um so the the number of tokens you were talking about is the number of tokens you

Gal Oren: talking

Tomer Bitan: are uh like um you need to pay for or is that the number of tokens that the model

Gal Oren: about.

Tomer Bitan: is receiving? Cuz it's it sounds kind of weird for you to pay for the entire context every time you prompt it given that it's the existing context of the conversation.

00:04:38

Tomer Bitan: No.

Erel Kaplan: Yeah. But because the the like the the how how it works,

Gal Oren: Good.

Erel Kaplan: the the like the the very essence of how the the agent works is that you give it one task and it goes with this task from from start to end and you don't like the the the additional input tokens is you're not giving any additional input tokens. It's just what the

model is generating plus the initial input tokens you

Tomer Bitan: Uh-huh. So,

Erel Kaplan: gave.

Tomer Bitan: so every time the model creates a new like call to itself, you have to pay for all the context that you previously received from the model again,

Gal Oren: for all the

Erel Kaplan: Yes. Yes.

Tomer Bitan: even though it's model generated tokens and you already paid for

Erel Kaplan: Yes.

Gal Oren: previously

Erel Kaplan: Yes.

Tomer Bitan: them.

Erel Kaplan: Yes. So, so with the Yeah.

Tomer Bitan: That's very weird.

Erel Kaplan: So with the with I I'm I'm not sure how exactly this is like as you can see here this is the the pricing for the GPT 5.1.1 C codeex max.

00:05:54

Erel Kaplan: So most of the the you see this is like one uh one one session. So as you can see the the total tokens is 3390,000 something and most of it is cached input tokens. So this is all all of these tokens is tokens from previous messages that that the model the the model cached and it used it again in the next turns. So this is like the the last turn.

Gal Oren: Ariel,

Erel Kaplan: Yeah.

Gal Oren: I I want you to do for next time, I want you to to do a test. Select 12 kernels that you think that are let's say the lassi kernels or something like that. Okay. And perform the following test.

Erel Kaplan: Mhm.

Gal Oren: Try to move the model end to end on those kernels with three different versions of of the GPT. The 5.1 the 5.2 and the 5.3. while while for each one you also change the

Erel Kaplan: Yeah.

Gal Oren: um the reasoning from the uh mini to the extra high. What I'm trying to what I'm trying to do here is for you to have for the paper a subset of a test in which you are showing that while you haven't tried all of the different pro uh options you do know to

quantify at least in a small test the reason why we are choosing 5.1 uh mini and I will tell you why because some reviewer can tell

00:07:47

Gal Oren: you that maybe if we would have switched to a better model it will take less tokens. Someone can also tell you that if we would increase the reasoning so the amount of time that the model would need to take to finish the work would maybe be shorter. So we need to showcase that given those different trends that we

Erel Kaplan: Yeah.

Gal Oren: find like some sort of an integral between those and and quantify how

Erel Kaplan: Yeah.

Gal Oren: much exactly for at least 12. Take it take the lassi subset for for how much how much do we know to how how much the difference is is is is been able to be measured. So we will be able to convince the the hyperparameter choice except for you know money and amount of parameters or things like that.

Erel Kaplan: Yeah. Yeah. So I I I did something uh it's not in the scale you're you're talking about,

Gal Oren: Okay.

Erel Kaplan: but I I did test it on one on one on one kernel with the

Gal Oren: Mhm.

Erel Kaplan: this is the Codex Mini.

00:09:06

Erel Kaplan: H this is here it took this is the same uh the same

Gal Oren: Uhhuh.

Erel Kaplan: job for the Codex Max and the Codex Mini.

Gal Oren: Uhhuh.

Erel Kaplan: So here we can see that it used the like extra session to uh it couldn't do uh in in one of the sessions it was not able to get a compiling uh code which triggered this exact same this

Gal Oren: What? What? What's the difference between the lines? What's the difference? I'm I'm using I'm

Erel Kaplan: is yeah so so this is here you can see like this is one only one power

Gal Oren: losing.

Erel Kaplan: cortex is uh three to four uh sessions. Yeah, because this Yeah. So this is here in the Codex Max. This is like an average. Yeah.

Gal Oren: Yeah, I got it.

Erel Kaplan: There's reason.

Gal Oren: I got it.

Erel Kaplan: Yeah.

Gal Oren: I got

Erel Kaplan: There's no reason that here I just put all of them.

Gal Oren: this.

Erel Kaplan: Yeah.

00:10:03

Erel Kaplan: So this is the CEX mini. So you can see that the tokens usage is is much higher here in some of the sessions. like this is 1 million and this is 3 million tokens and

Gal Oren: Mhm.

Erel Kaplan: uh yeah so it's still yeah it it is still cost less so this is like half half or maybe a little bit more than half the price of the CEX max yeah so but

Gal Oren: Mhm.

Erel Kaplan: I think that with the everything that every two days a new model comes out. I think that additional testing is needed because I I'm I I can't tell like if maybe like now the 5.3.3 can do it like in maybe

Gal Oren: You should you should have I mean no you should keep you should keep what you're currently doing as as it is.

Erel Kaplan: less

Gal Oren: It's okay. But have at least this small subset ready. to have like this graph shown in the paper because someone will ask about it. Someone will have like this question about why haven't you used this model and that one?

00:11:17

Gal Oren: Why haven't you used this parameter and that parameter even if the results will turn out to be better in a in a future model and with extra reasoning you at least be able to meas to to quantify by how much and you would say okay practically I cannot do

this because of cost but I at least I know I know to quantify how better it is for this task maybe it will turn out to be not not making any change and that will be also very interesting right because you you will you you will be able to showcase that most of the actions that codex perform in in this specific case doesn't really need a lot of reasoning it it need a lot of trial and error need a lot of uh I don't know it's it's maybe it's a

Erel Kaplan: Yeah.

Gal Oren: very code related problem and there is no nothing more than just like a lot of context and tokens so it's it's an open question in a

Erel Kaplan: Yeah.

Gal Oren: way

Erel Kaplan: Yeah. And

Niranjan Hasabnis: So just a question actually um I I think I was debating about the point that Tomar made right I think it's a very valid point where after every iteration the

00:12:26

Gal Oren: What

Niranjan Hasabnis: context size should should keep on increasing for the model right and with the context

Erel Kaplan: Yeah.

Gal Oren: is

Niranjan Hasabnis: increasing it it should have better chance of completing the task right getting maybe compilation or runtime correct right so but somehow the cache input tokens are not increasing like what should it be increasing All

Erel Kaplan: No, the no the this is like the the only the final final message I can show you.

Niranjan Hasabnis: right.

Gal Oren: check That's

Erel Kaplan: Maybe here I have some I have like

Gal Oren: it.

Erel Kaplan: full trace.

Niranjan Hasabnis: Let's go.

Erel Kaplan: Yeah. So this is a full full trace of the of of codeex. You can see like what what I copied to the slide is this this last last message. So here you can see like this is the cached sometimes it it goes like it does same same like a message is more than uh like one. You see here the cached input tokens is is the same here but here this is less. So so it does increase between uh between

00:13:42

Gal Oren: Love it.

Erel Kaplan: turns. Yeah. Um yes. So so this is this is how it goes. So I I really I I really tried to to change I downloaded the codeex repo and I tried to to mess with how it uses the history and how when the

Gal Oren: Okay.

Erel Kaplan: history some part of the history drops from the context and I was not able to get a good good results because it it sometimes If if you drop some of the context in some point,

Gal Oren: Yeah.

Erel Kaplan: it won't it a lot a lot of the time it won't even complete the task. And yeah, I was able to uh save some tokens because I removed some unnecessary tools from the original uh uh codeex and uh I changed the system instructions a bit.

Gal Oren: Next.

Erel Kaplan: So uh and this is all like if you let's say if this you you make it like 18,000 tokens this this will save you a lot because this is 18 times the the length of the session and not 20 times the length of the session.

00:15:10

Erel Kaplan: So even if you save here like 2k tokens this is this is a

Gal Oren: Mhm.

Erel Kaplan: lot.

Gal Oren: Okay. Uh, I I want to I want to Okay, that's that's great.

Erel Kaplan: Yeah.

Gal Oren: I want to to to to sums this up. So eventually you have this you have this uh um you have this under control. You're you're taking care of that point. Uh you also showed me that you're working on the uh interface

Erel Kaplan: Yeah.

Gal Oren: and so forth and extending to multi APIs

Erel Kaplan: Yeah. I can I Yeah,

Gal Oren: translation.

Erel Kaplan: I can I can show it in in in like two

Gal Oren: Okay. So yeah,

Erel Kaplan: minutes.

Gal Oren: go go go ahead do that because then I would want to uh I would I would want you in like one minute to also give like a brief about how the supercomputing paper about this will look like how we will wrap it up the whole thing.

Erel Kaplan: Okay. I'm I'm not sure I I I have right now in my head like how it will look at the at the end.

00:16:20

Gal Oren: So I I will do it for you in in that

Erel Kaplan: Okay. Okay. So, this is the current status of how it works.

Gal Oren: case.

Erel Kaplan: So, the uh there's a really beautiful Oops, that's not here.

Gal Oren: Okay.

Erel Kaplan: This is Yeah.

Gal Oren: Show show the video. Show the video. So I while you are showing the video I will say that what we want for the in contrast to the

Erel Kaplan: Okay.

Gal Oren: to the preprint that we have that we sent to ACL while I'm not even sure that ACL is the right venue for this but whatever uh the No, but it was great to you know it was great to have a deadline to put it somewhere. Uh that's I'm I'm I'm grateful that we managed to do that uh at the while uh uh uh back then. But the idea now is for supercomputing in less than two months uh the submission to have this system that translate from every API to every API while having the I would say the new uh uh baseline uh benchmark of Samyak in this Samyak I'm talking about only about the like the things that I sent you mostly.

00:17:40

Gal Oren: Yeah. not not the augmented part and so forth the to the collection of

Samyak Jhaveri: Okay.

Gal Oren: the um uh canonical API to API collection of benchmarks. So we will test those over the system and um while having this like almost as a product. So basically what we would like to do is to say we can translate from any language to any language

if you will give me the right amount of tokens and right amount of time. uh it's not it's not a full it will not be a foolproof because we

Erel Kaplan: Yeah.

Gal Oren: will not have at that time I guess we will not have the augmented uh version uh of the of the of the benchmark but we will have the collected benchmark that we talked about and

Erel Kaplan: No.

Gal Oren: um uh yeah Ariel go

Erel Kaplan: Yeah.

Gal Oren: ahead

Erel Kaplan: So this is like in really short this is like what I had in mind. So uh like this is like you choose from your from your computer. you you you like here I have a convolution 2D cuda and this is like the like the source code of the like if someone would want to translate the code this is like the source directory you're choosing the source API so this is CUDA and you're choosing the target API you can h insert a model and then you just start you can run uh with this multiple jobs at once and it tells you like where in in which part it is is analysis,

00:19:30

Erel Kaplan: translation, optimization and verification.

Gal Oren: Is

Erel Kaplan: H I'll stop it for now. And after after your job is done,

Gal Oren: it

Erel Kaplan: you can also analyze the performance. This is I don't have the I deleted the folders, but it it compares the original code versus the translated code. Yeah. So that's it.

Gal Oren: Very nice. Um, if we'll have time,

Tomer Bitan: Very cool.

Gal Oren: we will add features afterwards. But that's that's wonderful already. Yes,

Tomer Bitan: Yeah,

Gal Oren: Donna.

Tomer Bitan: if you're taking suggestions, I have an idea for a feature like

Gal Oren: Yes. Yes. Yes. Everyone that have suggestions for the for the UI,

Tomer Bitan: uh

Gal Oren: that's great because the idea is to have like a GitHub.io page with the paper. So reviewers will be able to jump into it and and virtually review it. And we will have also

like a print screen of the tool in the paper to like make sure that the reviewers know that this is like an actual working tool that they can have access to.

00:20:31

Gal Oren: That that would be like very important. Also, everybody is working on a on a skill um skill um uh file that virtually if they don't want to use the tool and they want to import the skill to let's say I don't know their own codeex um um uh local uh uh deployment. So, we will be able to do so. Yes, Tom.

Tomer Bitan: Oh, uh my suggestion was uh make a uploadable like a section for uploading tests. So if you want to make this a tool like you probably have a use case. So you come with your own test and your original code and if you want you have like whatever the original versions is and whatever internal verification you have there but also you can add in an additional test or two to like make sure that it's relevant to your specific use case and the whatever you're

Erel Kaplan: This is the the source code the the user is choosing the source code like this is like ah

Gal Oren: Nobody It says about the testing, not about the source

00:21:44

Erel Kaplan: yeah so I I'm not sure I fully

Gal Oren: code.

Erel Kaplan: understand

Tomer Bitan: it like so I was just mean like in addition to giving the source code please translate this code make sure it works on this specific test kind of thing you can maybe if

Erel Kaplan: yeah I got

Tomer Bitan: you if you're giving it the whole folder and it's looking for the tests in the folder maybe you just need to write make sure your tests are labeled slash test or thing, but just like any like uh just address the issue of how do I make sure that this thing

Erel Kaplan: Yeah.

Tomer Bitan: works for my needs kind of thing.

Erel Kaplan: Yeah.

Gal Oren: Mhm.

Tomer Bitan: Yeah, that's that's all

Gal Oren: Okay.

Tomer Bitan: I

Gal Oren: I I want to now before you we'll move to you Samak I want to move to you because I think it will be more you know harmonious to move to the benchmark part and then go to

Samyak Jhaveri: Sure.

Gal Oren: Tomer. So go ahead tell us uh where we are standing.

Samyak Jhaveri: After the meeting with U Arl Leon, Lean and Tor, I was able to get a better idea of what the benchmark should look like.

00:22:50

Samyak Jhaveri: I present I made a few slides. So, uh let to share my screen. Are you able to see my screen? Yes.

Gal Oren: Yes.

Samyak Jhaveri: Okay, perfect. Uh,

Gal Oren: It's It's a little bit It's a little bit grainy and

Samyak Jhaveri: so

Gal Oren: it's like It's like low. Yeah, it's uh it's like I I don't say it like sharply. I don't know why. Oh,

Samyak Jhaveri: I'm presenting the slide.

Gal Oren: now I see it sharply. Yes. Yes.

Niranjana Hasabnis: I think it takes a couple of uh

Gal Oren: Now, now it's good. Now it's good. Now it's

Samyak Jhaveri: Okay, good. Yeah, the internet is a little shaky,

Gal Oren: good.

Samyak Jhaveri: but anyway, um, so I found that the from the downloaded repositories, 35 of them were really good and I just got them. They have canonical explain uh examples. A lot of them have varying uh instructions to run those kernels. Some have cmake, make, some have nvc, nvc++.

00:23:54

Samyak Jhaveri: Only a few of them had actual cond usability. So uh that was a little

difficult but I focused on HC bench as the best example because that was the one that uh seemed most relevant for our purposes.

Gal Oren: What's going on?

Samyak Jhaveri: It has 506 kernels and it has so many translation pairs from LLM evaluations. So um this is the pipeline that I used to create uh a little example of the schema in the schema that we had decided.

Gal Oren: Mhm.

Samyak Jhaveri: So starting with survey and then analyzing the kernels and then designing the schema that took a few iterations and then I made the uh harness and the pipeline and then I tested it end to

Gal Oren: Uhhuh.

Samyak Jhaveri: end. Um little intro little more information about the downloaded benchmarks. There were 34 of 34 repositories served and 30 of them were high tier. Five of them were okay and there were 27 more than 27,000 implementations. Um so obviously the best ones were PRK Rodinia, Raha Perf, HCBench as expected. Uh some of the were many apps,

00:25:03

Gal Oren: Mhm.

Samyak Jhaveri: proxy apps and libraries.

Gal Oren: All

Samyak Jhaveri: We already saw that in Gal's uh graphical uh PNG uh picture. So for most of them so many of them were hybrid kernels in the sense that they used multiple

Gal Oren: right.

Samyak Jhaveri: um parallel programming APIs in the same program. Uh the most common ones were these like with CUDA with hip cuda with cycle cuda with openmmp um and these are the specific examples of those values.

Gal Oren: Fore

Samyak Jhaveri: Uh I will also send this this does not look good. I'm going to send this separately to the group. Um so taking note from Gal's visualizations we have this let's move over. So this is the main contribution from my end over the span of the last one one and a half weeks. Sorry I have not been able to work too much on this.

Gal Oren: on this side.

Samyak Jhaveri: I uh had pizza interviews. But um so my idea that I discussed with uh everyone on the call last time was having a manifest first like a directory where you could discover uh whether first of all a kernel to kernel a canonical translation exists or

00:26:10

Gal Oren: 45.

Samyak Jhaveri: not and then you could find out where exactly it exists in a specific file structure. An interesting thing that uh was pointed out last time was that in an ideal situation that it's just one kernel inside one sweet nice little C program and that just runs end to end. But that's not the way real real benchmarks work. There are other files,

Gal Oren: But

Samyak Jhaveri: helper files and um uh directory structures to navigate. So essentially uh one of the most important fields of this directory so to

Gal Oren: one of the most important

Samyak Jhaveri: speak is the exact location in the local uh machine where this

Gal Oren: Please

Samyak Jhaveri: exists. not in the local machine but like where exactly this particular file and folders exists and level two is the actual storage for that along with the metadata I would say metrics um speed up compilability metrics u performance metrics and so on I

Gal Oren: fantastic.

Samyak Jhaveri: can keep yeah so the main focus is on the level two structure which is the spec structure

00:27:30

Gal Oren: And I'm and I'm making sure that this structure and JSON is compatible with a real and timer right.

Samyak Jhaveri: Yes, I

Gal Oren: Tomer do you have anything to are you are you able to digest

Samyak Jhaveri: use

Gal Oren: this into your own testing? Uh I'm I'm making sure that's like the whole idea for the time being.

Erel Kaplan: Yeah.

Gal Oren: Yeah.

Tomer Bitan: I mean,

Gal Oren: Very good.

Tomer Bitan: it'll take a tiny modification,

Gal Oren: Okay.

Tomer Bitan: I guess.

Gal Oren: Okay. Okay. Go go go ahead

Samyak Jhaveri: Okay. Um,

Gal Oren: Samyak.

Samyak Jhaveri: so in the paper we could include a condensed version of this spec and uh like a listing and the schema itself can go in a separate repository release along with uh the rest of the files. So um the example one that I populated with this particular schema has performance metrics has hardware specifications for my uh 4070. Um so this is really critical for the paper's credibility in the sense that um it controls what the LLM sees during evaluation.

00:28:48

Samyak Jhaveri: So in the email nine pointed out that the uh LLM should never be able to see the test the uh the test suites and the expected

Gal Oren: Yeah,

Samyak Jhaveri: results.

Gal Oren: that's that's Ne's comment, right?

Samyak Jhaveri: Yes. Uh and that was a very strong point in the design of the schema. So the prompt payload is the only thing that is sent to the LMS for translation. So for example for ncuda uh that's nearest neighbors and it's supporting files like anyh files any cu files and that's all um the other supporting files are make files cmake files uh and so on no other information about the performance no other information about um the memory usage from the previous run is given to the LLM of course and for verification it

Gal Oren: Mhm.

Samyak Jhaveri: is com the final values from the test are compared with the expected values that are derived from the previous one. Um but that is only after the LLM has generated the code we've profiled it using the profiler and we get a final set of results.

00:30:00

Samyak Jhaveri: Only then for correctness the values are matched.

Gal Oren: Look at this.

Samyak Jhaveri: Of course it is expected that the values may vary a little bit because of non-determinism uh I mean um parallel computing and um uh difference in the hardware so the floating

Gal Oren: Yeah, it's it can be in some range.

Samyak Jhaveri: points some

Gal Oren: You can you can define a range to to that.

Samyak Jhaveri: yeah exactly

Gal Oren: It's also okay.

Samyak Jhaveri: um so it took like seven six seven drafts I

Tomer Bitan: I have a small

Samyak Jhaveri: will send you the exact draft.

Tomer Bitan: question.

Samyak Jhaveri: I can show you right show it to you right

Gal Oren: So you already you already have like those 16 60 pairs from each API to each

Samyak Jhaveri: here. Oh no,

Gal Oren: API.

Samyak Jhaveri: not not all 60 pairs. I only did for HC

Gal Oren: uh for HSC bench.

Samyak Jhaveri: bench.

Gal Oren: But for HC bench, you already have like the all of the pairs with everything that Okay.

Samyak Jhaveri: Yeah.

Gal Oren: So before you will move on,

00:30:57

Samyak Jhaveri: Yeah.

Gal Oren: I want to just make a u a comment here that I want you to deliver this specific hackbench test to a rail as soon as possible. So you will be able to first also contribute to our current paper and not only to the uh powerbench paper pair which you'll be your own uh first uh author on uh that's a first but I want also that to be a part in the supercomputing paper that we're trying to write. So basically what it means is that uh you will take uh this all structure for hackbench and uh turn it into the cornerstone of the uh um all to all translation in the um in the new paper. So if if I will need to to stress the new paper is about translating from whatever API to whatever API but when you say whatever API to whatever API you need to come up with okay what are the whatever APIs that you have that you choose so then we can take from let's say from this

presentation everything up to up to the uh conclusion of why we are choosing the following APIs because we conducted the survey and so forth.

00:32:28

Gal Oren: morph and then say that for the time being we are taking the hackbench example maybe we will add some others but we'll take the hackbench example with I don't know four to five five APIs to five APIs and we will showcase the results on on top of that because we are not trying in in the SE paper to keep claiming only about openmp it's not it's not interesting only about openmpp it will be the serial the openmpp uh the CUDA sec and what else do we have there?

Samyak Jhaveri: in Hegbench.

Gal Oren: Ah there is heap but we don't need heap.

Samyak Jhaveri: We have

Gal Oren: I I think he no one no one uses um but but

Samyak Jhaveri: Yeah,

Gal Oren: that

Tomer Bitan: I have a small question.

Samyak Jhaveri: I I

Gal Oren: yes

Samyak Jhaveri: think

Tomer Bitan: uh when you mentioned you're not uh inputting the the support files I think oh cuz I

Samyak Jhaveri: I I am

Tomer Bitan: thought you said uh that they are not shown to the LM like the

Samyak Jhaveri: Oh, no. the the result files are not shown to the LLM like the like the result expected output is not

00:33:34

Tomer Bitan: build Oh

Samyak Jhaveri: shown to the LLM.

Tomer Bitan: okay

Samyak Jhaveri: They're only used after the LLM writes the code, it runs profiles and we get results.

Tomer Bitan: but but the build infrastructure before the original one is shown.

Samyak Jhaveri: Yes, it's shown the make files, the headers. Oh,

Tomer Bitan: Okay.

Samyak Jhaveri: wait. That's not that's that's on me. This is wrong.

Tomer Bitan: Okay. Okay. Yeah. Got

Niranjan Hasabnis: So are you guys constructing this manually like step by step or how are you is this a automated process

Samyak Jhaveri: I've tried to automate most of the steps here.

Gal Oren: manually.

Niranjan Hasabnis: okay I mean the

Gal Oren: I don't think anyone writes code anymore.

Samyak Jhaveri: Uh

Gal Oren: I I don't think that

Niranjan Hasabnis: cons itself right you have to look at the data set and then understand which files

Tomer Bitan: They started doing that

Gal Oren: exists.

Niranjan Hasabnis: contain the verification check and all those things right it's not a Um,

Tomer Bitan: after.

Niranjan Hasabnis: it's not easy to do this automatically.

00:34:35

Niranjan Hasabnis: I mean,

Samyak Jhaveri: That's true. So I built this harness for uh doing this and it basically is a CLI where CLI interface where I can just add in a new benchmark by using certain commands and it does all these things where um it loads the JSON specs if it's possible resolves the parts and then it substitutes the variables to build a clean pipeline. and then it executes. I ran everything on my personal GPU. Um, so the speed up metrics might be a little modest honestly, but I think it's just to show that it works. And finally, we get a JSON export and a formatted output. So, for example, for this one, I could actually run this one. Uh and uh these are the results that I

Gal Oren: Okay.

Samyak Jhaveri: got.

Gal Oren: So I I'm I I want to have I want us to also have time for Tomer. So my concluding point here is that that's that's great work and you are on the right path. Uh I want you to work closely with Ariel over the over the next uh weeks.

00:35:53

Samyak Jhaveri: Mhm.

Gal Oren: Uh so all of that will be already part of the of the new SC paper. um you will be able to that's imagine imagine that this is like the new section of testing in in that paper in contrast to the previous open MP1 that we had and what's important about it it's not just like we are taking Hackbench is that we are explaining how to construct the the uh I would say this the new JSONs the new framework the new uh template to how um how test will be digested by the paracordic system. Okay. Is that is that is that okay by you?

Samyak Jhaveri: Yeah, I'll work with Arrow on this. I'm noting it

Gal Oren: Any

Samyak Jhaveri: down.

Erel Kaplan: Uh yes.

Gal Oren: comments?

Erel Kaplan: So this is the we talked about it when we when we met. I think that was already two weeks ago about the now the like the what I'm doing with the paracordex is um taking the necessity of the JSON files off. So if if you're a developer that wants the your code in a different API, you don't need a specific uh JSON or something.

00:37:36

Erel Kaplan: you you you would use the the maybe the like the the other JSON like not the first one. But if you have a source folder with the with your code that you want uh transferred to different API or even maybe

Gal Oren: Thank

Erel Kaplan: just uh enhance its performance then that's all you need for now for for parodex to

Gal Oren: So I would I would suggest you and Syak having a followup on this. It's very important that you will have uh a close connection on on this because this is the thing that I want to be integrated in the paper. I want us to have because eventually think about it eventually it will be a separate

Erel Kaplan: Okay.

Gal Oren: paper with all of the things about this benchmark and you don't want then to

Erel Kaplan: Yeah.

Gal Oren: you know to redo everything to prove that power codex can be compat compatible with that benchmark. So we need to find a common ground between the two the two works.

Erel Kaplan: Yes.

Gal Oren: So everything that will be continuously be done for for the for the uh for the benchmark will be you know as automatically as possible be digested by paracodex for the you know for its next version.

00:39:02

Gal Oren: So even though we are now talking about like the most basic version of it it's still critical that that that will be you know easily digested by the current system and will be finally be able to be described in the paper. So that's that's that's a point that I want to stress eventually if we want if we really want to submit to SC we need that part being like finished in like at most two weeks from

Samyak Jhaveri: What else?

Gal Oren: now okay because then you will have a lot of testing to do to and probably they will figure out some problems and then we will move to the describe and so forth. So I'm I'm stressing that we will need this at least in two weeks from now. So keep keep an open channel Y and Samyak and try to to to you know to close the loop as soon as

Samyak Jhaveri: Yeah. Um,

Gal Oren: possible.

Samyak Jhaveri: I still need to be added to the Slack or Discord channel. Uh so that would really help me communicate better and

00:40:01

Gal Oren: Yes. Yes. Nanjan please uh if you can uh yeah

Samyak Jhaveri: faster.

Niranjana Hasabnis: Yeah, I

Gal Oren: you will be added and um uh also change uh yeah the discord and everything with

Niranjana Hasabnis: will.

Gal Oren: Samyak directly. You don't you don't also need the group thing. You can work uh onetoone uh on this or WhatsApp or whatever you want to to work on. I don't I don't

know. Uh okay, that's great. I want to uh move for the last part for Tomer for an update on the more uh uh I would say the new frontier in this. So Tom, go ahead.

Tomer Bitan: Yep. Uh,

Le Chen: Yeah,

Tomer Bitan: where

Le Chen: if you're sorry to go um um I may have run uh

Gal Oren: Yes.

Tomer Bitan: is

Le Chen: um for another meeting. Um I have quick some quick comments. Um I saw the first part regarding talking about the tokens. So um are we um are we are we do we have anything like with a memory uh memory issue like the considering the memory control over the over the like the flow of the of the agents.

00:41:08

Le Chen: Do we have anything that in mind?

Gal Oren: Uh, I'm not sure what memory means. Can you can you

Le Chen: Uh for example,

Gal Oren: elaborate?

Le Chen: we we have a we have those u like u um where we have a verification error right and uh we see that's uh that didn't give a pass we send it back to some previous agent.

Gal Oren: Right.

Le Chen: So what kind of the like the uh information we we send along with that um back sending back forwarding message. Do we include the for example the whole purpose like messages or do we include only the the the the previous verification discussions among

Gal Oren: That's a real Samyak.

Le Chen: the

Gal Oren: Please go ahead and answer.

Erel Kaplan: Yeah. So, so like the you mean like if the if one of the one one instance of the agent is not able to uh get its job done basically what's the Yeah. So, so currently what I'm doing is like giving the like there's a a summary of the session of the agent that it sums up what what

00:42:15

Gal Oren: Foreign

Erel Kaplan: it did what didn't go right. So this alongside the corrupted code, the code that doesn't work goes to the like the retry and yeah, so there's no like only the corrupted code and the and the and the the message uh from the last session and h yeah it works fine for now. Yeah. So if it doesn't succeed on the first try, then it goes to second try and most of the time it does well.

Le Chen: Uh okay so I heard two messages. First is uh you mentioned that you have a summary of the what's what's been going so far right? So is that summary generated by by a large empty model or how how it's oh it's it's basically feeling some like or generating based on some format you're given or basically you give all the all the all the current status to logic model to give a summarization

Gal Oren: something

Erel Kaplan: It's the the it's generated by the by codeex. It's like a mechanism. It has like it generates like a a final message uh that it gives summary message.

00:43:35

Gal Oren: Please

Erel Kaplan: I I think it it called and uh this message goes to the retry session.

Le Chen: I see. I see. Um yeah, I I I think we we could have put some work like how these uh different uh messages like where we we parse these memory and stop there and send it back to the to the to the to another agent or previous agent to fix the current issue. that could be interesting and some ablations could also be done on that. Um I I I also want to mention that if you guys uh uh have some problems with uh uh

Gal Oren: Very

Le Chen: token usage I could help if you can provide me the code I can run without uh worrying about the tokens at inside the lab. So um yeah if if you have problem with that let me know.

Gal Oren: good.

Erel Kaplan: Thank you.

Gal Oren: Stay stay in touch with Le on this. Maybe it will be able to really foster the work because it will be able to do more testing and so forth.

00:44:42

Erel Kaplan: Yeah.

Gal Oren: Okay, great great contribution. Uh, bye for now. Bye-bye, Tom. Go

Tomer Bitan: Yeah. Um, so Mike is fascinating.

Gal Oren: ahead.

Tomer Bitan: We don't have too much to cover. Um, so I I kind of we're still very much in the building phase, so we don't have uh nice results to show yet. Uh,

Gal Oren: Mhm.

Tomer Bitan: but I kind of broke the the plan down into four phases um that I'm looking at. And we kind of did we're basically in phase two and hopefully we can get done with that by probably not all next week but in in two weeks max we should be done with phase two.

Gal Oren: Mhm.

Tomer Bitan: Um the first phase is is getting the reasoning model um the basic main model uh to run. It's it's uh so it comprised of trying to run the the

Gal Oren: That's

Tomer Bitan: coconut or HRPO and the the I told everybody was in the meeting two weeks ago that I found uh a really new paper that was retracted but one of

00:45:50

Gal Oren: not

Tomer Bitan: the interesting point like they had some other point but everybody noted that the interesting point in the paper is this little realignment matrix um that they talked about. So basically um the problem with the latent reasoning was that it was hard to scale it up and I ran a little test and these are like more or less um in pure accuracy results they are more or less comparable. Uh and the realignment matrix is the easiest one to implement. Um,

Gal Oren: What happened?

Tomer Bitan: so after fighting a lot with uh the memory out of memory on the the mutant cluster and having Claude Opus betray me a bunch of times, adding extra memory leaks and taking me a while to find them. Um, I'm like almost uh got everything working. Like I got all the comparisons done and I'm almost ready with like a latent reasoning model. Um, I'm not

Gal Oren: Do you have to uh already uh did you talk to Nean about the machine that you need for the final model and so forth?

00:46:59

Gal Oren: What we talked like two weeks

Tomer Bitan: Yeah. So, yeah. So,

Gal Oren: ago.

Tomer Bitan: when I was doing like when I when I was I was going to message Neoan and then I realized that for this part since we're only trying to run a 70 billion model um I should be able to run it on the Technon cluster. So like for creating the data set and I will only need the large machine when we finally get to phase three which is

Gal Oren: Mhm.

Tomer Bitan: run the the 70 billion along with the the other model.

Gal Oren: Mhm.

Tomer Bitan: So sorry to phase four.

Gal Oren: Okay.

Tomer Bitan: So for now I think I'm I should be fine as long as I can I had a small problem running um the the like a 70 billion model um fully sharded like for inference there.

Gal Oren: Uhhuh.

Tomer Bitan: the existing code base was not suited for that because they were all working with small models. So I I think I should be done with that in like a day or two.

00:47:45

Gal Oren: Mhm. Mhm.

Tomer Bitan: Um and in parallel uh Lean and Roy have been working on the second part. So Lean is working on adding LLM as a judge and the variable comparison with and between her tests. And correct me if I'm wrong,

Gal Oren: Mhm.

Tomer Bitan: Lan, but you said you should think you'd be done by before next week,
lian ghrayeb: by Friday.

Tomer Bitan: right? Before this by Friday.

lian ghrayeb: Yeah. Yeah.

Tomer Bitan: Okay, wonderful.

lian ghrayeb: Yeah.

Tomer Bitan: Yeah. So, that's like almost fully done.

Gal Oren: Amazing.

Tomer Bitan: Um, and uh I did a lot of research into the the PRM. So, how we want to train a PRM and what what does the data need to look like? Um I I didn't add the graph here which is a mistake but basically what we're trying to do is create like um have a problem uh and then a few different runs of um the what's it called rollouts of the the reasoning. So you have a problem a reasoning and then from each reasoning step you generate a few

00:48:47

Gal Oren: Mhm.

Tomer Bitan: other for the tests of the reasoning. So it's like a little tree. Um and then all the final results confirmed for each reasoning step how good it was. So basically we're generating labels for each reasoning step based on all the different variations of code generated beyond that reasoning step. Um and so is finishing um to to to make the multiple

Gal Oren: Mhm.

Tomer Bitan: reasoning queries and uh adapting the low evaluation framework that he showed us last time into like how do I also save the latent vectors uh and that kind of stuff and bolting the the final model to that and the few little touches that we needs to do.

Gal Oren: Mhm.

Tomer Bitan: I'm hoping that we can be done with that around the end of next week and have

Gal Oren: Mhm.

Tomer Bitan: like a full data set ready by then end of next week like after a few bugs we'll probably find and all that kind of stuff. So hopefully two weeks we're already past phase two.

Gal Oren: People

Tomer Bitan: Um and off to like real work in phase three which is having the the big data set and starting to train uh the little guidance model uh which is called a PR.

00:50:04

Tomer Bitan: Yes, John.

Niranjan Hasabnis: So I was just confused about data set like how what data set do you require for

Tomer Bitan: Uh so basically to train the PRM what we saw that everybody's doing is having

Niranjan Hasabnis: this

Tomer Bitan: uh a list of thinking steps having a a score for each of them and then training the little guidance model on how good is is this thinking step like a like a what's it called?

Niranjan Hasabnis: off.

Tomer Bitan: uh regular training I don't know what's it called u so

Niranjan Hasabnis: Turn off. Is it okay?

Tomer Bitan: the the plan was to take hebench or pal palatans the the the

Gal Oren: Mhm.

Tomer Bitan: hackbench like subset that we used for for uni and have a big

Gal Oren: Thank you.

Tomer Bitan: LLM that's the one I'm finishing on now generate the thinking tokens and the

Gal Oren: Thank you

Tomer Bitan: results for that then we score everything with the with the score that Lean

Gal Oren: to

Tomer Bitan: built and make a little data set of um here are latent vectors scored according to how good the final result is for

00:51:07

Gal Oren: Okay, we need now to wrap it up.

Tomer Bitan: the

Gal Oren: But that's that that's great progress, Tomer. I like it. Make sure to as as you said to start,

Tomer Bitan: Yeah.

Gal Oren: you know, the more the deeper testing in the following two weeks.

Tomer Bitan: Yeah.

Gal Oren: I also added a reminder that that's probably if this is going to be like a very interesting research. So that can go to new rips and the deadline for that going to be

estimated around May. So think think about that.

Tomer Bitan: Yeah. Okay.

Gal Oren: Have like your own deadline and keep keep track on your own deadline.

Tomer Bitan: Perfect.

Gal Oren: Uh but but eventually great progress to Ariel Samyak Lean.

Tomer Bitan: Yeah.

Gal Oren: I I liked everything that I saw today.

Tomer Bitan: Very small point.

Gal Oren: Yes.

Tomer Bitan: Uh and Lean are not invited to this meeting automatically. So I don't know if it's Gal or Nan,

Gal Oren: Please,

Tomer Bitan: whoever is in charge of that.

Gal Oren: please send them uh the the the link to this.

00:51:56

Gal Oren: Okay.

Niranjan Hasabnis: All right.

Gal Oren: Uh you can send it all also over email and um yeah and and

Niranjan Hasabnis: Yeah.

Gal Oren: and great we will we will keep track uh on both ends. We have the so virtually we have the uh SC in the next two months. We have then the new rips in the next three months from now. And then we will have eventually we'll have when we will have the paper of Samyak we will put it to the next venue. We will need to keep to think when when it's going to be appropriate because it's going to be like a little bit too too far from now. Uh but great great progress everyone. Um and uh we will see you soon in two weeks from now and keep keep the chat working and keep keep everyone updated on the things that you are doing.

Niranjan Hasabnis: Yeah. So I did share the link to the discord uh ch channel.

Tomer Bitan: Wonderful.

Niranjan Hasabnis: So I think lean you're not there in the channel. In case you want to join the link is there in the chat or I can also share the link over

Tomer Bitan: And oh,

Niranjan Hasabnis: email. Okay let me share it here.

Tomer Bitan: the Discord channel. Yeah. Okay. Thanks.

Transcription ended after 00:53:21

This editable transcript was computer generated and might contain errors. People can also change the text after it was created.